



INDIVIDUAL ASSIGNMENT

LEVEL 06

COMP60010

Enterprise Cloud And Distributed Web Applications

SENG23A1

Hand In Date: 11 May 2026

Name: K T Dilanka Yasuru

CB Number: CB011162

INSTRUCTION TO CANDIDATES

- 1. Students are advised to underpin their answers with the use of references (cited using the Harvard Referencing Style).**
- 2. Late submission will be awarded zero (0) unless extenuating circumstances (EC) are upheld.**
- 3. Cases of plagiarism will be penalized.**
- 4. The assignment should be submitted as a softcopy:**
 - a. The softcopy of the written assignment and source code must be uploaded to given link in the LMS**

Table of Content

1. Introduction.....	4
2. Current Infrastructure Analysis and Cloud Justification.....	5
2.1. Justification for AWS Adoption.....	5
3. Cloud Migration Strategy.....	8
3.1. Phased Cloud Migration Strategy.....	8
3.2. Infrastructure-as-Code (IaC) Automation Snippet.....	9
4. Target AWS Architecture Design.....	13
4.1. Multi-AZ VPC Design and Security Boundaries.....	13
4.2. Customer Web Portal (CWP).....	14
4.3. Smart Meter Data Platform (SMDP).....	14
4.4. Data Storage and Analytics Platform.....	15
4.5. Observability, Security, and Disaster Recovery.....	15
5. Security, Governance and Compliance.....	16
5.1. Multi Layered Security Architecture.....	16
5.2. Network Security using Security Groups and NACLs.....	16
5.3. AWS WAF and Shield.....	16
5.4. Logging and Governance using CloudTrail, AWS Config, and Security Hub.....	17
5.5. Encryption using AWS KMS (At Rest and In Transit).....	17
5.6. GDPR Considerations: Data Residency, Access Control, and Audit Trails.....	18
5.7. Sample IAM Policy JSON.....	18
6. Cost Optimisation and Support Strategy.....	23
6.1. Estimated AWS Costs.....	23
6.2. Recommended Cost Optimisation Techniques.....	24
7. Data Migration and Continuity.....	25
7.1. Low-Downtime Data Migration Strategy.....	25
7.2. Historical Data Archiving using Amazon S3.....	25
7.3. Data Validation Techniques.....	26
8. Hybrid Connectivity and Edge Integration.....	27

8.1. Secure Hybrid Connectivity Design.....	27
8.2. Edge Data Integration using AWS IoT Core and Greengrass.....	27
9. Innovation Using AI and Analytics.....	29
9.1. IoT Data Streaming with AWS IoT Core and Kinesis.....	29
9.2. AI-Driven Anomaly Detection and Forecasting using Amazon SageMaker.....	29
9.3. Visualising Insights with Amazon QuickSight.....	31
10. Practical Implementation & Evidence.....	32
10.1. VPC Creation.....	32
10.2. Lambda, API Gateway Deployment.....	33
10.3. IoT device simulation.....	34
10.4. CloudWatch metrics and CloudTrail logs.....	36
10.5. API Testing.....	38
10.6. Infrastructure as Code Deployment.....	39
11. Conclusion.....	40
12. References.....	41

1. Introduction

AquaSense Smart Utilities Ltd (ASU) is a major South Asia company that is making smart water and energy metering solutions accessible to millions of households, factories and municipal councils. With data from more than 30,000 real time sensors expected to increase, ASU's on premises infrastructure has its own limitations: It's hard to scale during peak periods, and it doesn't have the analytics capabilities needed. To help overcome these capacity limitations, ensure regulatory compliance, and gain predictive insights, ASU has strategically decided to move to Amazon Web Services (AWS).

This document provides an in depth Cloud Architecture and Migration Plan to modernize ASU's technological capabilities. Scrutinously examines the shortcomings of existing infrastructure and aligns them to scalable resilient AWS solutions. In addition, it outlines a staged cloud migration approach that follows AWS Well-Architected Framework cloud architecture principles to the design of a strong target AWS architecture. The goal of the transition is to have 95% service availability, process telemetry data in near real-time, lower operational costs by at least 35% and make AI-driven demand forecasting possible, and future proof its smart utility services.

2. Current Infrastructure Analysis and Cloud Justification

ASU has a legacy on premises VMware cluster that currently hosts around 180 virtual machines, a centralized monolithic web app, and MySQL/PostgreSQL databases. This infrastructure has served the company well in the past, but the mandate to process telemetry data from more than 30,000 smart meters in real time has highlighted some key weaknesses in a number of areas.

Scalability: No automatic scaling, no infrastructure as code, monolithic architecture. This forces ASU to make manual estimates of required capacity and fails to adapt its capacity dynamically during busy periods, causing the potential for degraded service.

Resilience: ASU's architecture is very susceptible to failures due to the fact that there is no disaster recovery (DR) site and that the architecture relies on a manual backup strategy. The monolithic approach to the app is the single point of failure and the ability to achieve the business objective of 95% service availability for customer facing applications is extremely challenging.

Performance: Limited monitoring and log visibility results in poor proactive performance tuning and troubleshooting on the infrastructure. Additionally, the current infrastructure does not offer any advanced analytics or artificial intelligence features, and ASU is unable to create predictive insights or demand forecasts.

Security and Compliance: Manual and limited logs make it very difficult to ensure strong security, governance and auditability. This presents a major challenge for regulatory compliance and GDPR alignment, and for this reason, data residency, access control and audit trails must be closely managed.

Cost: These datacenters require large upfront capital investments in infrastructure and the hosting of idle assets, costing ASU more money. The company needs to cut at least 35% in infrastructure and operational costs, something that is challenging when utilizing physical servers that need to be racked, stacked and maintained.

2.1. Justification for AWS Adoption

The migration to AWS will enable ASU to move from a monolithic on premises architecture to a highly available, event driven, microservices based architecture. The following is a mapping of ASU's challenges to specific AWS services.

2.1.1. Solving Scalability Challenges

Amazon EC2 and Auto Scaling: Guessing capacity and battling during peak traffic, ASU can automatically add and remove virtual servers according to real-time demand with Amazon Elastic Compute Cloud (EC2).

ASU can use AWS Lambda to process the massive amount of telemetry data it is receiving from 30,000+ meters. This is a serverless compute service that will run code in response to events, increasing or decreasing as data volume increases or decreases, with ASU not having to provision or manage any underlying servers.

2.1.2. Addressing Resilience Challenges

Amazon RDS: ASU can migrate its on-premises MySQL and PostgreSQL databases to Amazon Relational Database Service (RDS) or Amazon Aurora. Managed services simplify time consuming and repetitive activities such as hardware provisioning, patching, and backups, and offer Multi-AZ replication for high availability and automated failover.

Amazon Route 53: Amazon Route 53 offers highly available and scalable Domain Name System (DNS) routing to give customers a consistent experience when accessing the Customer Web Portal, with health checks to ensure that customers will always be directed to healthy endpoints.

2.1.3. Addressing Performance Challenges

Amazon CloudFront: To improve the performance of the Customer Web Portal (CWP), ASU can utilize Amazon CloudFront. By caching data, applications, and APIs nearer to the end users, this Content Delivery Network (CDN) guarantees secure delivery of content to all the world with extremely low latency and high transfer speeds.

Amazon Simple Storage Service (S3): Amazon S3 can be used as the basis of ASU's data lake and store historical data and IoT telemetry for fast and durable storage while enabling powerful AI and analytics applications such as Amazon SageMaker and QuickSight.

2.1.4. Addressing Security and Compliance Issues

Improved security with AWS IAM and KMS: Least privilege access and MFA can be effectively implemented with AWS Identity and Access Management (IAM). AWS Key Management Service (KMS) will simplify key management to encrypt all data at rest and in transit, helping meet the GDPR requirements.

AWS WAF, Shield and CloudTrail: Amazon CloudFront integrates with AWS WAF (Web Application Firewall) to protect against common web application exploits (such as SQL injection attacks) and integrates with AWS Shield to mitigate DDoS attacks. AWS CloudTrail

is now bringing full log visibility for comprehensive API call history to address the challenge of lack of log visibility for strict governance and auditing.

2.1.5. Solving Cost Challenges

ASU is shifting to a pay-as-you-go model by moving to AWS, eliminating fixed capital costs. Up to 72% savings on compute usage can be achieved with AWS Savings Plans and AWS Compute Optimizer can recommend the best resources to lower AWS costs. Moreover, Amazon S3 lifecycle policies will automatically apply to older telemetry data to lower its cost to a more affordable tier, so that ASU can comfortably achieve its 35% cost reduction driver.

3. Cloud Migration Strategy

Due to the constraints of ASU's legacy on premises infrastructure, the following phased cloud migration plan was developed to follow the AWS Well Architected Framework.

3.1. Phased Cloud Migration Strategy

- **Discovery and Application Assessment**

Firstly, a thorough evaluation of the current 180 VMs and monolithic apps will be conducted. Use the AWS Application Discovery Service to gather configuration, usage, and behavior information from the on premises VMware cluster. This data will be exported, which will be used to calculate the Total Cost of Ownership (TCO) and applications for migration prioritization. AWS Migration Hub will be used to centrally track all migration progress.

This point corresponds with the Cost Optimization pillar and allows ASU to accurately inform the data and avoid over-provisioning resources, only paying for what it consumes.

- **Selection of Migration Approaches**

From the discovery information, ASU will determine which workloads will be categorized for migration, following these migration strategies.

Rehost (Lift and Shift): AWS MGN will automatically convert the source servers to run natively on AWS, reducing manual, time-consuming processes.

Replatform (Lift and Reshape): Existing MySQL and PostgreSQL databases will be replatformed to Amazon Aurora and Amazon RDS, bringing cloud optimizations, and minimizing database administration overhead.

Refactor (Re-architect): Centralized monolithic Web application and IoT telemetry processing systems will be refactored to leverage the full potential of cloud-native microservices, enhancing agility, performance and scalability.

- **Pilot Migration**

ASU will conduct a pilot migration of a sample of its data and applications, to reduce risk.

ASU will use AWS Database Migration Service (AWS DMS) to safely move a test data base. The source database will be available throughout the process to eliminate as much downtime as possible to applications that depend on it. ASU will monitor the metrics and progress of this pilot in one place through AWS Migration Hub.

This phase can test the migration process safely and shows that the system is capable of functioning correctly and consistently, thus contributing greatly to the Reliability pillar.

- **Modernisation using Containers and Serverless Services**

This phase transforms the monolith architecture into an agile and highly scalable one. The Customer Web Portal will be broken up into microservices running as containers, orchestrated with Amazon Elastic Container Service (Amazon ECS) with AWS Fargate. ASU will use an event driven architecture comprised of AWS Lambda, Amazon SQS, and Amazon SNS to deal with near real-time data from 30,000+ smart meters.

Managed services and serverless reduce the need to manage physical servers. It also supports the Sustainability pillar by making sure to use resources to the fullest and distribute the environmental footprint of shared data center parts throughout the AWS Cloud.

- **Production Rollout, Validation, and Rollback Planning**

The final phase provides for a smooth and hassle-free customer transition.

ASU will perform a Canary or Blue/Green deployment with AWS CodeDeploy in order to incrementally move user traffic to the new AWS environment. Amazon CloudWatch will be heavily used by the post-deployment stage, which will be the only time when the system metrics, logs, and business KPIs are monitored in a proactive manner.

Rollback Plan: If any critical errors are discovered during validation, an immediate rollback will be performed, bringing traffic back to its original environment without any issues.

This follows the Operational Excellence design principles of "anticipate failure" and "frequent small reversible changes" where by if this deployment fails, it can be easily reversed to reduce the effect on the customer.

3.2. Infrastructure-as-Code (IaC) Automation Snippet

The following Terraform snippet provisions the target architecture (Multi-AZ VPC, private and public subnets, replatformed RDS database, and ECS Cluster for microservices) and demonstrates how to automate this migration approach.

```
terraform {
  required_version = ">= 1.5.0"

  required_providers {
    aws = {
      source = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}
```

```

provider "aws" {
  region = "ap-south-1"
  alias  = "primary"
}

# 1. Networking : Multi-AZ VPC Design with Public and Private Subnets
resource "aws_vpc" "main" {
  cidr_block      = "10.0.0.0/16"
  enable_dns_support = true
  enable_dns_hostnames = true
  tags = {
    Name      = "asu-main-vpc"
    Environment = "production"
    Project    = "AquaSense Migration"
  }
}

resource "aws_subnet" "public_az1" {
  vpc_id            = aws_vpc.main.id
  cidr_block        = "10.0.1.0/24"
  availability_zone  = "ap-south-1a"
  map_public_ip_on_launch = true

  tags = {
    Name = "asu_public_subnet_1"
  }
}

resource "aws_subnet" "private_az1" {
  vpc_id            = aws_vpc.main.id
  cidr_block        = "10.0.2.0/24"
  availability_zone  = "ap-south-1a"

  tags = {
    Name = "asu_private_subnet_1"
  }
}

resource "aws_subnet" "private_az2" {
  vpc_id            = aws_vpc.main.id

```

```

cidr_block      = "10.0.3.0/24"
availability_zone = "ap-south-1b"

tags = {
  Name = "asu_private_subnet_2"
}
}

# 2. Security: Network Security Groups using Attachment Resources
resource "aws_security_group" "ecs_sg" {
  name          = "asu-ecs-sg"
  description   = "Security group for ECS microservices"
  vpc_id        = aws_vpc.main.id

  tags = {
    Name = "asu_ecs_sg"
  }
}

resource "aws_security_group_rule" "ecs_ingress_https" {
  type          = "ingress"
  description   = "HTTPS inbound from VPC"
  from_port     = 443
  to_port       = 443
  protocol      = "tcp"
  cidr_blocks   = [aws_vpc.main.cidr_block]
  security_group_id = aws_security_group.ecs_sg.id
}

# 3. Compute: ECS Cluster for Microservices (Fargate)
resource "aws_ecs_cluster" "smdp_cluster" {
  name = "asu-smdp-cluster"

  tags = {
    Name          = "asu_smdp_cluster"
    AppPurpose    = "iot_telemetry_processing"
  }
}

# 4. Data Storage : Primary Amazon RDS Database

```

```

resource "aws_db_subnet_group" "default" {
  name          = "asu-db-subnet-group"
  subnet_ids    = [aws_subnet.private_az1.id, aws_subnet.private_az2.id]

  tags = {
    Name = "asu_db_subnet_group"
  }
}

resource "aws_db_instance" "primary" {
  provider          = aws.primary
  allocated_storage = 20
  engine            = "postgres"
  instance_class    = "db.t3.micro"
  identifier        = "asu-primary-db"
  db_subnet_group_name = aws_db_subnet_group.default.name
  vpc_security_group_ids = [aws_security_group.ecs_sg.id]
  skip_final_snapshot = true

  username = "dbadmin"
  password = "AsuSecurePassword123!"
}

```


Private Database Subnets: Protect the database back end servers (Amazon Aurora, Amazon DynamoDB), and the caching layer (Redis), which are always completely separated from Internet access.

VPC Endpoints: Gateway endpoints for Amazon S3 and DynamoDB, and Interface endpoints for AWS KMS, Secrets Manager, and CloudWatch keep local data flow to the AWS managed services within the AWS private network.

4.2. Customer Web Portal (CWP)

The CWP provides usage dashboards and alerts and reporting to ASU customers.

Perimeter Security and Routing: Amazon Route 53 routes User traffic to Amazon CloudFront, which is protected from DDoS attacks and Web exploits by AWS WAF and AWS Shield.

API & Load Balancing: REST API endpoints are handled by Amazon API Gateway. The Web traffic goes through the Internet Gateway (IGW) to an Application Load Balancer (ALB).

The ALB routes requests to an Amazon ECS (Fargate) cluster where the web portal microservices run. The ECS services are scaled up and down automatically in response to customer demands.

Data Storage: The CWP uses a highly available Amazon Aurora relational database (primary in AZ1, failover in AZ2) for transactional data, and Redis Cache for session management.

4.3. Smart Meter Data Platform (SMDP)

The SMDP is designed to ingest and process near real-time telemetry from 30,000+ smart meters.

Edge and Ingestion: Data is securely collected at the edge using AWS IoT Greengrass gateways and transmitted to AWS IoT Core via the MQTT protocol.

Event Driven Processing: The IoT Rules Engine routes incoming telemetry to an Amazon SQS Queue, decoupling ingestion from processing. An Event Processor AWS Lambda function pulls messages from the queue for processing. Unprocessable messages are caught by Dead Letter Queues (SQS DLQ and Lambda DLQ) for troubleshooting.

Alerting Workflow: If the Event Processor detects specific anomalies, it triggers an event-driven workflow using Amazon SNS. It publishes to an Event SNS topic, which

triggers an Alert Evaluator Lambda function. This function then pushes alerts to a Notification SNS topic to notify users.

4.4. Data Storage and Analytics Platform

The architecture incorporates specialized storage solutions to support advanced analytics and predictive capabilities powered by AI.

Hot Storage: The Event Processor Lambda stores near real-time telemetry data directly to Amazon DynamoDB.

AI and Data Lake: Telemetry data is streamed through Amazon Kinesis Data Firehose to an Amazon S3 Data Lake at the same time. AWS Glue catalogs the data, which is used by Amazon SageMaker to create demand forecasting and anomaly detection models.

Visualization: Amazon QuickSight integrates with the S3 Data Lake, OpenSearch, and Aurora to create business intelligence reports and dashboards.

4.5. Observability, Security, and Disaster Recovery

Monitoring: Amazon CloudWatch collects metrics and logs from ECS and Lambda and distributes them for monitoring. CloudWatch pushes logs to Amazon OpenSearch to enable enhanced log analytics and to an S3 Log Archive bucket.

Governance & Compliance: The architecture uses AWS Security Hub and AWS Config to help manage the security posture continuously. AWS KMS encrypts Aurora and DynamoDB data at rest and AWS Secrets Manager securely injects the database credentials into the ECS services.

Hybrid Connectivity and Migration: AWS Direct Connect and a VPN Gateway provide secure hybrid connectivity from ASU's on-premises network. To help move historic MySQL and PostgreSQL data to Aurora and DynamoDB, AWS DMS (Database Migration Service) is deployed.

Backup: AWS Backup is responsible for automatic backups for Aurora, DynamoDB and S3 Data Lake with a 1-hour RPO and a 4-hour RTO.

5. Security, Governance and Compliance

AquaSense Smart Utilities (ASU) will use a Multi Layered, Defense in Depth (DID) security architecture that follows the AWS Well-Architected Framework for the Smart Meter Data Platform (SMDP) and Customer Web Portal (CWP).

5.1. Multi Layered Security Architecture

IAM, Users, Roles, Least-Privilege Policies and MFA – IAM's identity foundation will be developed and created according to the principle of least privilege, where users and systems will only be granted the permissions required to complete their task.

All AWS users in the workforce will be granted access to the AWS environment via a single identity provider with temporary credentials, and Multi-Factor Authentication (MFA) will be strictly enforced for all human identities in the AWS environment, especially the AWS account root user.

Systems like Amazon EC2, ECS and AWS Lambda will use IAM roles to get short-term security credentials, to reduce the exposure of long-term credentials. The AWS IoT Core credential providers will be used in IoT smart meters to ensure authentication and short-term, scoped access.

5.2. Network Security using Security Groups and NACLs

ASU's multi-AZ VPC will include both stateless and stateful inspection to isolate the network.

Security Groups: Security Groups will be used to enforce the traffic flow between instances with tight control, they are stateful, instance level firewalls. For instance, the Amazon Aurora database Security Group will be set up to only accept traffic from an ECS microservices Security Group, denying any other traffic.

Network Access Control Lists (NACLs): NACLs will be used to allow or deny traffic at the subnet level on the public, application and data subnets, and will be stateless.

5.3. AWS WAF and Shield

Perimeter security will be implemented at the edge to protect external assets such as the Customer Web Portal and API endpoints from external attacks.

AWS WAF: AWS WAF will be configured to monitor HTTP/HTTPS requests that are sent to Amazon CloudFront and the Application Load Balancer. It will thwart common web exploits and attack patterns, including SQL injections and cross-site scripting (XSS), that could be used to compromise ASU's applications.

AWS Shield: AWS Shield offers automatic inline detection and mitigation to protect against Distributed Denial of Service (DDoS) attacks and ensure availability of ASU's customer-facing applications.

5.4. Logging and Governance using CloudTrail, AWS Config, and Security Hub

The visibility and auditing will be provided continuously via automation of monitoring and governance solutions.

CloudTrail: CloudTrail will be turned on so that it can capture all AWS API calls that are made throughout the account. Records callers, call time, client source IP, and request parameters, for in-depth security analysis and compliance auditing.

AWS Config: Config will continuously monitor and document the configurations of the AWS resources used by ASU. It allows for automatic assessment of a resource to desired security states and alerts when a resource falls out of compliance (such as a database being made public).

Security Hub: Security Hub will serve as a security posture management service to manage security posture across the cloud. It will also consolidate security alerts from different services and continually run security best practices from the AWS Foundational Security Best Practices, among others.

5.5. Encryption using AWS KMS (At Rest and In Transit)

Data protection mechanisms will be used to ensure confidentiality and integrity of ASU's telemetry and customer data.

Encryption at Rest: AWS Key Management Service (KMS) will generate and manage cryptographic keys to encrypt data at rest in Amazon S3, Aurora, and DynamoDB.

Encryption in Transit: Encryption will be required for all network traffic that carries sensitive data into, out of, and within the VPC, ending at the Elastic Load Balancing (ELB) and/or API Gateway, with a secure protocol such as HTTPS/TLS.

5.6. GDPR Considerations: Data Residency, Access Control, and Audit Trails

In accordance with the General Data Protection Regulation (GDPR), ASU's architecture follows the principles of data governance.

Data Residency: ASU will keep 100% control over their data on AWS. Data in the AWS Region selected will be stored in that Region and not transferred or replicated to another AWS Region unless specifically provided by ASU to meet GDPR data residency requirements.

Access Control: IAM's fine-grained access control together with attribute-based access control (ABAC) ensures that access to Personally Identifiable Information (PII) is only granted to authorized personnel and systems as required.

GDPR compliance: AWS CloudTrail gives complete and comprehensive audit trails of access and changes to infrastructure and data, which is required by GDPR's data processing logs requirement.

5.7. Sample IAM Policy JSON

The following Terraform configuration is an example of the principle of least privilege to the Smart Meter Data Platform (SMDP). These definitions limit the access of AWS Lambda and AWS IoT Core to only the resources required to process smart meter telemetry instead of granting access to the entire resource.

The access to required resources is explicitly provided for the event processing functions in this role, which is assumed by the AWS Lambda Service. It binds an AWS managed policy for standard CloudWatch Logging, as well as a custom policy that prevents access to the telemetry table, Data Lake S3 bucket, SQS queues (telemetry and dead letter queues) and SNS topic for alerts.

```
# Lambda Execution Role
resource "aws_iam_role" "lambda_execution" {
  name = "${var.project_name}-lambda-execution-role"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Action = "sts:AssumeRole"
```

```

        Effect = "Allow"
        Principal = {
            Service = "lambda.amazonaws.com"
        }
    }
]
}))

tags = {
    Name = "${var.project_name}-lambda-execution-role"
}
}

# Attach AWS managed basic execution policy (CloudWatch Logs)
resource "aws_iam_role_policy_attachment" "lambda_basic" {
    role      = aws_iam_role.lambda_execution.name
    policy_arn =
"arn:aws:iam::aws:policy/service-role/AWSLambdaBasicExecutionRole"
}

# Custom policy - DynamoDB, S3, SQS, SNS access
resource "aws_iam_role_policy" "lambda_custom" {
    name = "${var.project_name}-lambda-custom-policy"
    role = aws_iam_role.lambda_execution.id

    policy = jsonencode({
        Version = "2012-10-17"
        Statement = [
            {
                Sid      = "DynamoDBAccess"
                Effect    = "Allow"
                Action    = [
                    "dynamodb:PutItem",
                    "dynamodb:GetItem",
                    "dynamodb:Query",
                    "dynamodb:Scan",
                    "dynamodb:UpdateItem"
                ]
                Resource = [
                    aws_dynamodb_table.telemetry.arn,
                    "${aws_dynamodb_table.telemetry.arn}/index/*"
                ]
            }
        ]
    })
}

```

```

    },
    {
        Sid      = "S3DataLakeAccess"
        Effect   = "Allow"
        Action   = [
            "s3:PutObject",
            "s3:GetObject",
            "s3:ListBucket"
        ]
        Resource = [
            aws_s3_bucket.data_lake.arn,
            "${aws_s3_bucket.data_lake.arn}/*"
        ]
    },
    {
        Sid      = "SQSAccess"
        Effect   = "Allow"
        Action   = [
            "sqs:ReceiveMessage",
            "sqs:DeleteMessage",
            "sqs:GetQueueAttributes",
            "sqs:SendMessage"
        ]
        Resource = [
            aws_sqs_queue.telemetry.arn,
            aws_sqs_queue.telemetry_dlq.arn
        ]
    },
    {
        Sid      = "SNSPublish"
        Effect   = "Allow"
        Action   = [
            "sns:Publish"
        ]
        Resource = [
            aws_sns_topic.event_topic.arn,
            aws_sns_topic.notification_topic.arn
        ]
    }
]
}))
}

```

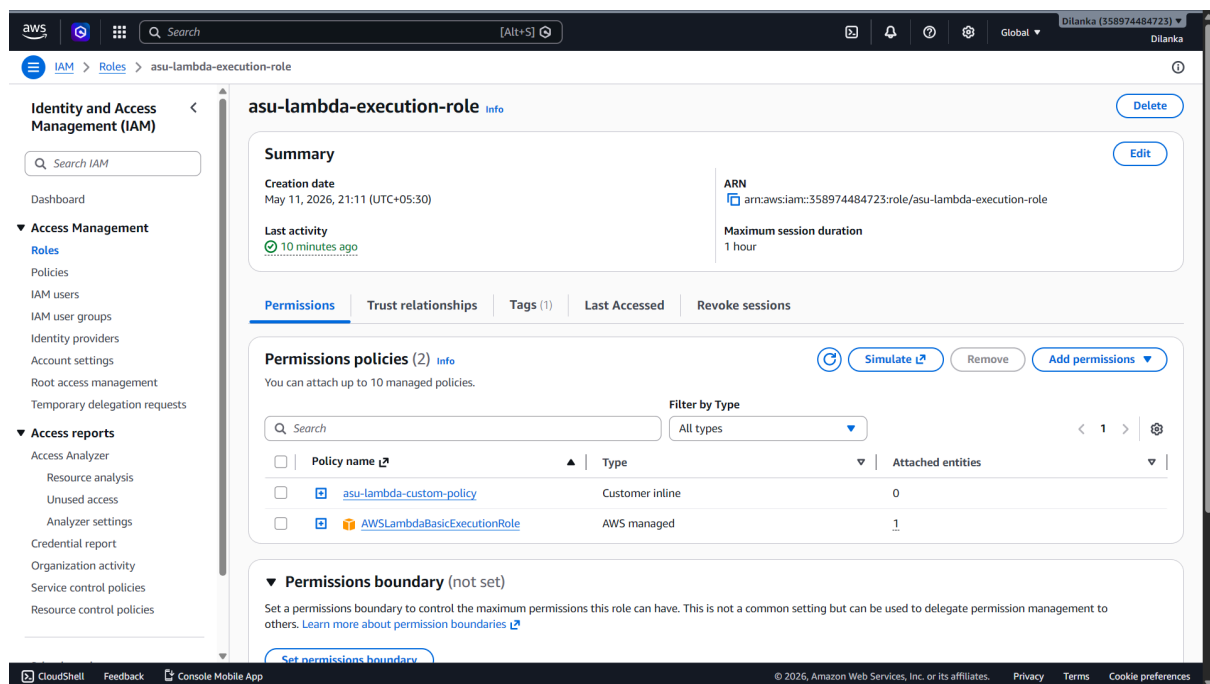


Figure 2: Sample IAM Policy JSON Terraform Configuration.

IoT Core Rule Role and Policy

This role enables the AWS IoT Core service to use it to run AWS IoT rules. The attached custom policy prevents the IoT rule engine from sending anything but messages to the specified SQS telemetry queue, ensuring that the data ingested from the edge is securely sent to the queue for processing without provisioning of anything else.

```
# IoT Core Rule Role
resource "aws_iam_role" "iot_rule" {
  name = "${var.project_name}-iot-rule-role"

  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Action = "sts:AssumeRole"
        Effect = "Allow"
        Principal = {
          Service = "iot.amazonaws.com"
        }
      }
    ]
  })

  tags = {
```

```

    Name = "${var.project_name}-iot-rule-role"
  }
}

# IoT Rule Policy - Send to SQS
resource "aws_iam_role_policy" "iot_sqs" {
  name = "${var.project_name}-iot-sqs-policy"
  role = aws_iam_role.iot_rule.id

  policy = jsonencode({
    Version = "2012-10-17"
    Statement = [
      {
        Sid      = "AllowSQSSend"
        Effect   = "Allow"
        Action   = [
          "sqs:SendMessage"
        ]
        Resource = aws_sqs_queue.telemetry.arn
      }
    ]
  })
}

```

6. Cost Optimisation and Support Strategy

The architecture largely leverages managed, right sized, and serverless compute in order to achieve ASU's business requirement to reduce infrastructure costs by 35% or more. The monthly and annual costs were estimated with the help of AWS Pricing Calculator in Asia Pacific (Mumbai) region.

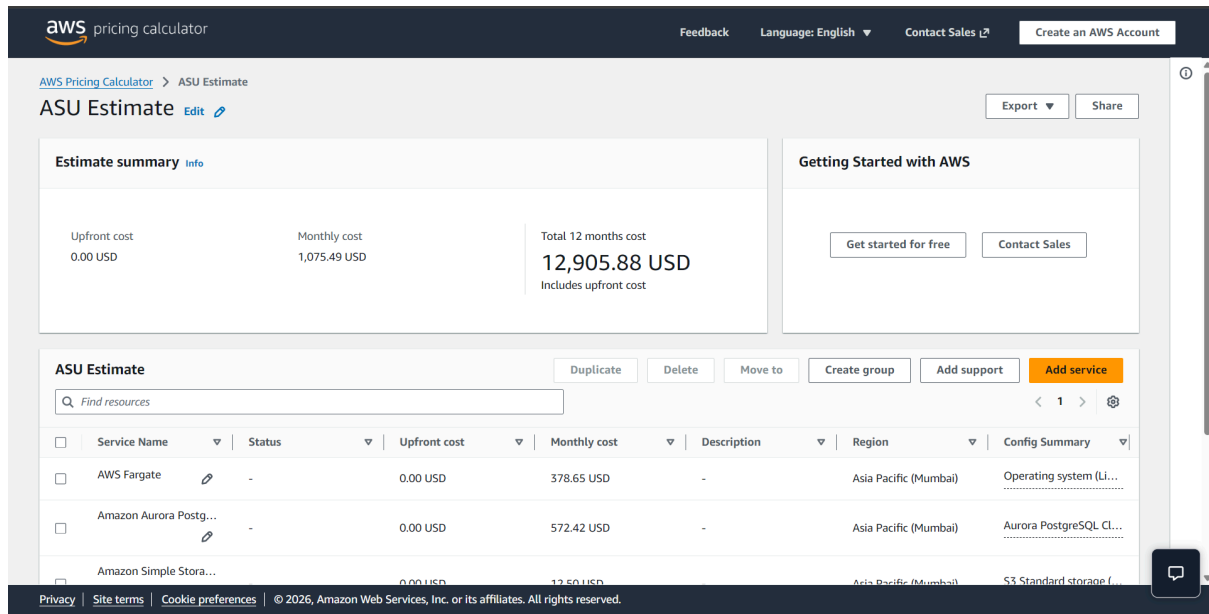


Figure 3: Estimated AWS Costs Breakdown.

6.1. Estimated AWS Costs

The projected monthly AWS cost is \$1,075.49, which totals \$12,905.88 for 12 months with \$0.00 upfront costs. The nature of the breakdown of key services is as follows.

Compute (Amazon ECS / AWS Fargate): \$378.65 / month.

The same 10 tasks/pods running Linux (x86 architecture) continuously for 730 hours/month. Each task gets 2 GB of memory and 20 GB of ephemeral storage. Fargate removes the need to worry about the underlying EC2 instances and ASU only pays for the exact amount of compute time required by the containers.

Database (Amazon Aurora PostgreSQL-Compatible): \$572.42 / month.

2 On-Demand instances (db.r6g.large) with 100% utilization to provide Multi-AZ high availability. The cluster has 500 GB of Aurora Standard storage.

Storage (Amazon S3): \$12.50 / month.

500 GB of S3 Standard storage for Data Lake and archiving of historical data.

Data Transfer: \$111.92 / month.

1 TB of internet data transfer out per month and no data transfer in.

6.2. Recommended Cost Optimisation Techniques

ASU should perform the following techniques to continually control use and minimize the \$1,075.49 baseline.

Compute Savings Plans: The price estimate currently takes On-Demand pricing, and ASU can sign up for a 1 Year or 3 Year Compute Savings Plan to save up to 72% on Fargate and Lambda costs.

S3 Storage Classes: S3 storage classes offer the lowest storage cost for various access patterns. ASU should implement S3 Lifecycle rules to move older, historical smart meter telemetry data to lower cost tiers of S3 storage, namely S3 Standard-Infrequent Access (S3 SIA) for infrequent access and Amazon S3 Glacier Flexible Retrieval (S3 Glacier Flexible Retrieval) for archiving.

AWS Compute Optimizer: AWS Compute Optimizer is a free service that leverages machine learning to analyze past utilization metrics. To avoid over-provisioning and to get automated recommendations on the right-sizing of Aurora size instances and Aurora Fargate size, ASU should regularly review their instance classes and Aurora task sizes using Compute Optimizer.

7. Data Migration and Continuity

7.1. Low-Downtime Data Migration Strategy

The plan is to migrate AquaSense Smart Utilities' (ASU) on-premises MySQL and PostgreSQL instances to AWS with minimal disruption to the service by using AWS Database Migration Service (AWS DMS) to achieve a near zero downtime migration.

This approach involves a phased strategy with the use of the Full load and change data capture (CDC) migration type.

Initial Full Load: AWS DMS will extract the existing database tables to the target Amazon Aurora or RDS databases in a bulk manner.

Change Data Capture (CDC): The full load is moving data, and any other transactions or changes that occur on the on-premises databases are captured and cached by the DMS Replication Instance.

Continuous Replication and Cutover: After the initial load is finished, AWS DMS applies the changes in the cache and starts replicating in real-time. Once the source and target are synchronized at a steady state, ASU can schedule a small window of time to pause on-prem traffic and let the last few transactions in flight drain to AWS, and then switch over traffic to the newly created AWS databases.

7.2. Historical Data Archiving using Amazon S3

Older telemetry information from the smart meters will be moved directly to Amazon Simple Storage Service (Amazon S3) for optimal size, cost and performance of the new operational databases.

AWS DMS now supports S3 as a target for the endpoint.

The historical data will be downloaded and saved in the S3 bucket as Comma-Separated Value (.csv) or Apache Parquet (.parquet) file format. Parquet is particularly suggested for analytical queries that are quicker and have a compact columnar storage structure.

This data can then be effortlessly combined with the AWS analytics and AI solutions to support business intelligence and compliance retention needs for data stored in S3.

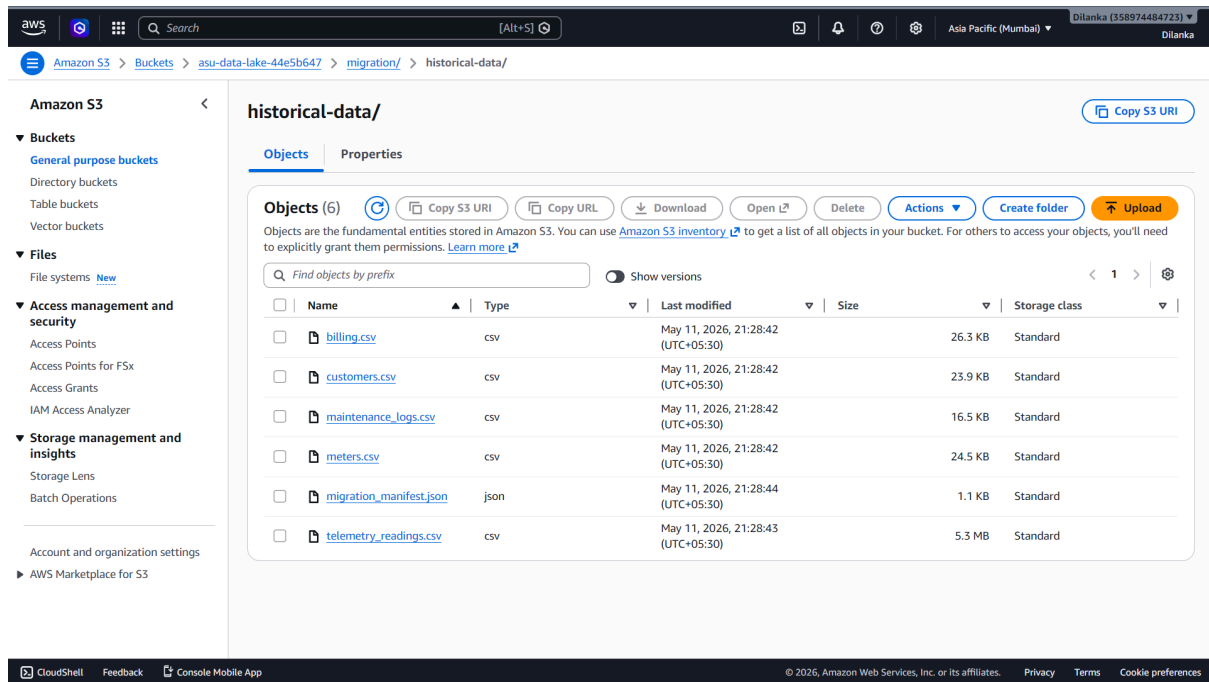


Figure 4: Low-Downtime Data Migration Strategy with AWS DMS.

7.3. Data Validation Techniques

It is important to have data integrity between the on-premises environment and AWS. The following validation mechanisms will be used.

AWS DMS Built In Data Validation: For the relational databases, data validation will be enabled at the task level. When the full load for a table is done, AWS DMS will automatically check the information row-by-row from the source MySQL/PostgreSQL database to the target AWS database to ensure that it matches exactly.

AWS Glue Jobs and Checksums: AWS Glue jobs will be used to validate the data for historical data migrated to Amazon S3. Glue can store the S3 data and execute automated scripts to compare row counts, perform cryptographic checksums and ensure data integrity and consistency against the source systems prior to decommissioning the legacy systems.

8. Hybrid Connectivity and Edge Integration

Adopting a hybrid networking and edge computing design to allow AquaSense Smart Utilities (ASU) existing on-premises infrastructure to communicate securely and efficiently with the AWS Cloud, for robust ingestion of more than 30,000 smart meters.

8.1. Secure Hybrid Connectivity Design

To connect the on-premises VMware cluster to the newly deployed AWS environment, ASU needs a reliable hybrid network. This will be done by creating redundant, highly available connectivity with AWS Direct Connect and AWS Site-to-Site VPNs.

Primary Connection (AWS Direct Connect): AWS Direct Connect dedicated network connection will be provisioned. This creates private, high-bandspeed and low-latency connectivity AROUND the public internet. This way, any data migrations or on-premises database synchronization (using AWS DMS) will be secure and reliable and of large scale.

AWS Site-to-Site VPN: AWS Site-to-Site VPN will be used for redundant backup (failover) in the event of an AWS Direct Connect link failure. This VPN will establish an encrypted IPsec tunnel across the public internet to end at a Virtual Private Gateway (VGW) connected to the ASU multi-AZ VPC.

8.2. Edge Data Integration using AWS IoT Core and Greengrass

ASU will be using Edge Computing to process the massive amount of data the 30,000+ smart meters are sending out, before sending the processed data to the cloud.

AWS IoT Greengrass (Edge Gateway): AWS will deploy edge gateways, locally, near the smart meters, and install AWS IoT Greengrass on them. Greengrass brings cloud services to local devices, letting them aggregate, filter, and take actions based on the telemetry data without having to connect to a cloud service. This decreases latency, bandwidth usage and allows critical alerts (such as high pressure loss) to cause local action even if there is no internet connection available.

The telecommunications data from the Greengrass edge gateways is securely sent to AWS IoT Core by MQTT protocol, and it is filtered and aggregated. The IoT Core is the cloud hub that allows easily managing billions of messages. The AWS IoT Rules Engine then easily

transitions the telemetry data to other AWS services for further processing or analysis, such as Amazon SQS, AWS Lambda and Amazon DynamoDB.

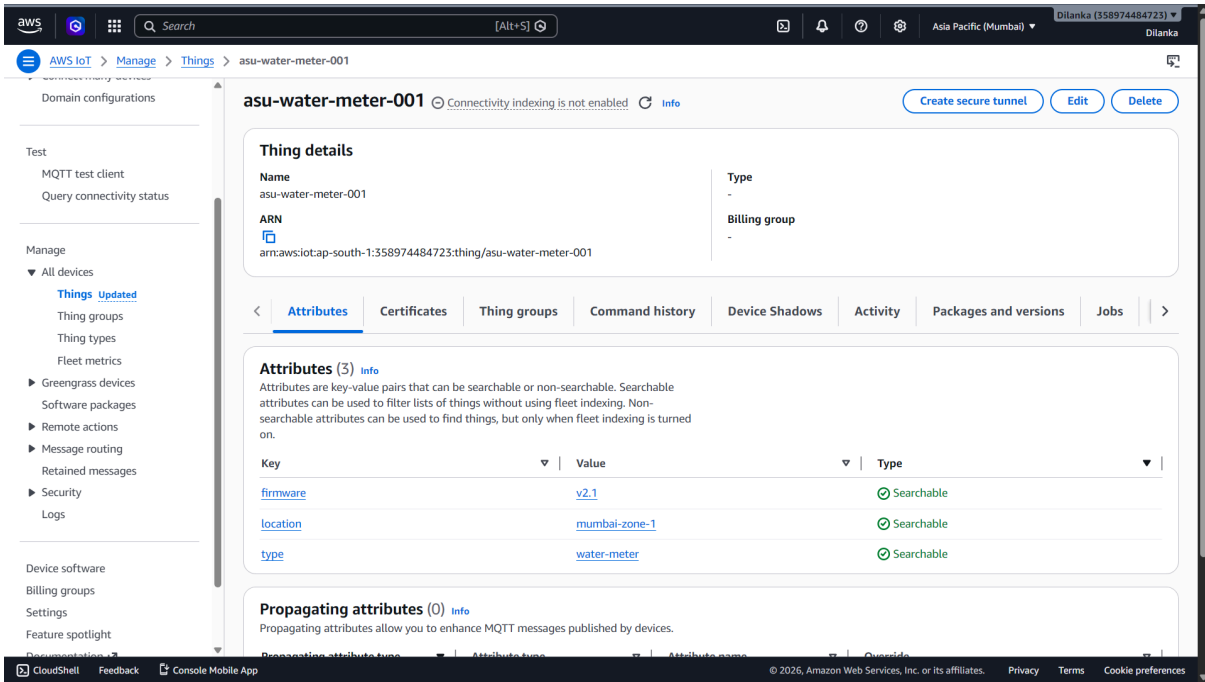


Figure 5: Secure Hybrid Connectivity Design with Direct Connect and VPN.

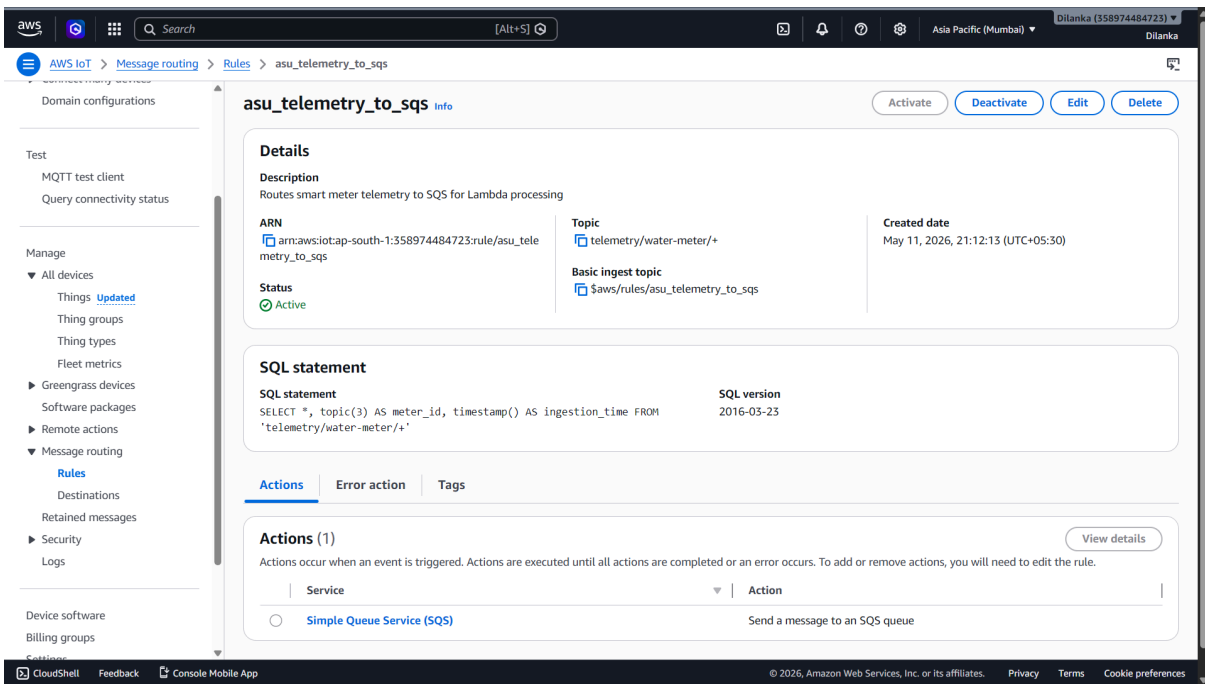


Figure 6: Edge Data Integration using AWS IoT Core and Greengrass.

9. Innovation Using AI and Analytics

We will use AWS managed AI and analytics services to move AquaSense Smart Utilities (ASU) from a reactive monitoring company to a proactive data-driven company. This architecture provides predictive insights, demand forecasting and real time anomaly detection without the burden of managing underlying infrastructure.

9.1. IoT Data Streaming with AWS IoT Core and Kinesis

The telemetry data from ASU's 30,000+ smart meters comes to AWS IoT Core via MQTT. We use the AWS IoT Rules Engine to analyze data messages and automatically send them to Amazon Kinesis Data Firehose to store and process it for a long time. Kinesis Firehose serves as a dependable streaming delivery channel, and compresses and transforms the JSON telemetry into the columnar, compressed, Apache Parquet format before streaming it into the central Amazon S3 Data Lake.

9.2. AI-Driven Anomaly Detection and Forecasting using Amazon SageMaker

With historical and near real time telemetry securely stored in Amazon S3, ASU can use Amazon SageMaker to build, train, and deploy machine learning models.

Anomaly Detection: ASU can detect abnormal water flow patterns or sudden drop in pressure that may be caused by pipes bursting or leaking using SageMaker's built-in algorithms including Random Cut Forest.

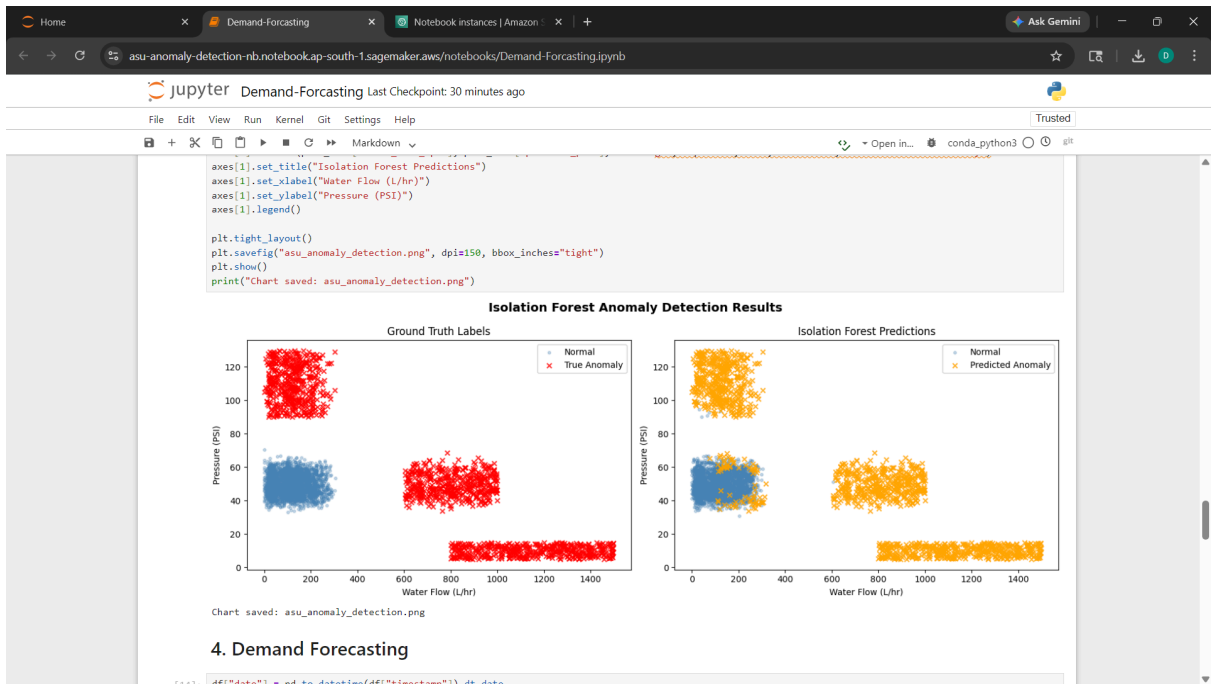


Figure 7: AWS SageMaker for AI-Driven Anomaly Detection.

Demand Forecasting: This is also where SageMaker will come into play by training models to relate past water/energy consumption with weather, time of day, season, etc. to accurately predict peak demand cycles and optimize accordingly water/energy distribution.

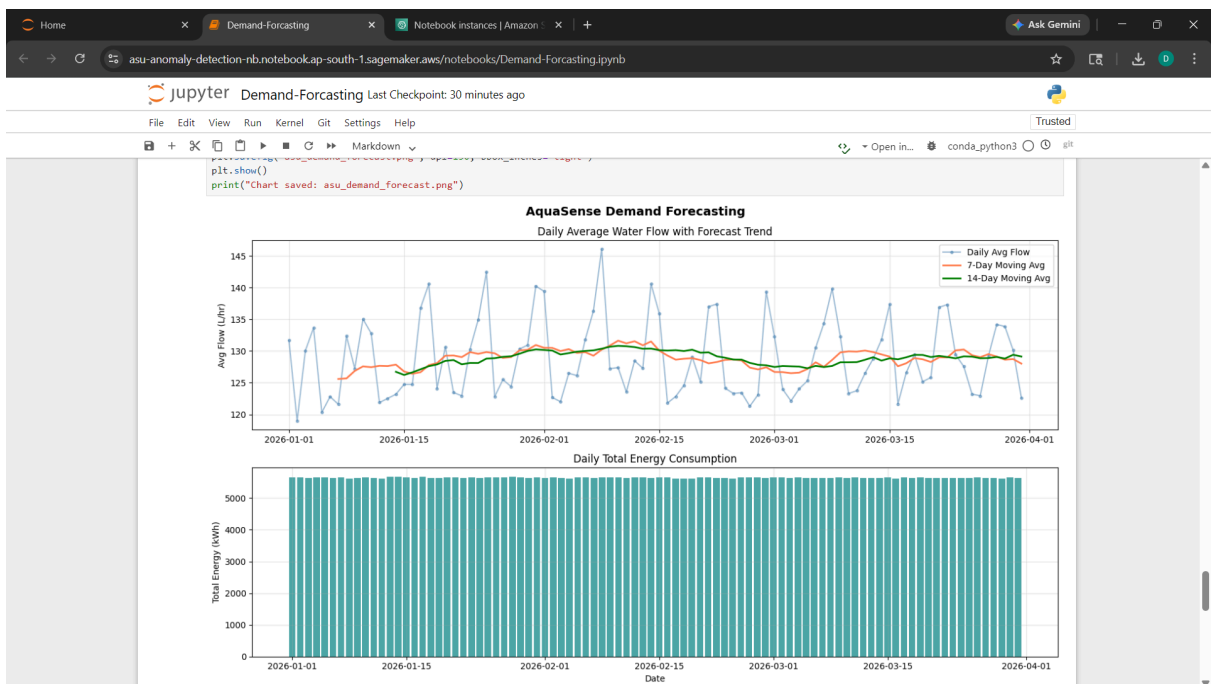


Figure 8: AWS SageMaker for Demand Forecasting.

9.3. Visualising Insights with Amazon QuickSight

We will deploy Amazon QuickSight to help these AI-driven insights become easy to access for ASU operations teams and municipal clients. QuickSight will be seamlessly integrated directly with the S3 Data Lake (via Amazon Athena) and SageMaker endpoints to create interactive, serverless Business Intelligence (BI) dashboards. These dashboards will show the real-time health of the network, identify areas of the network with detected anomalies, and show predicted consumption patterns.

10. Practical Implementation & Evidence

10.1. VPC Creation

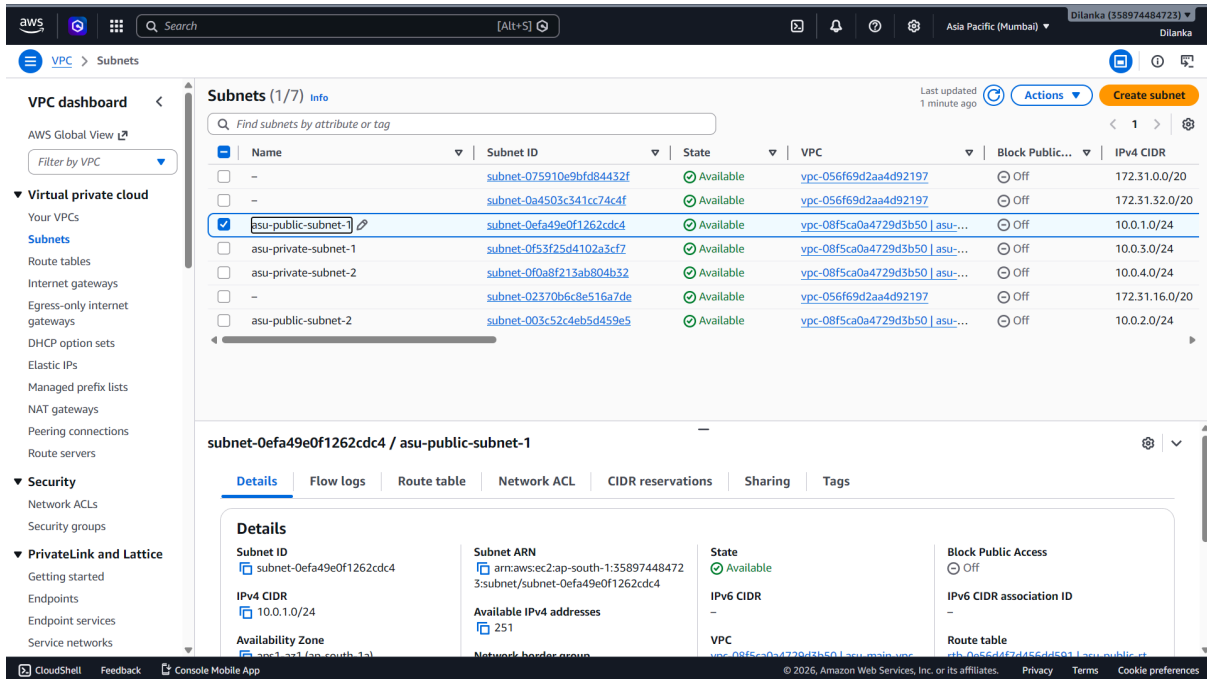


Figure 9: Practical Implementation - VPC Creation (1/2).

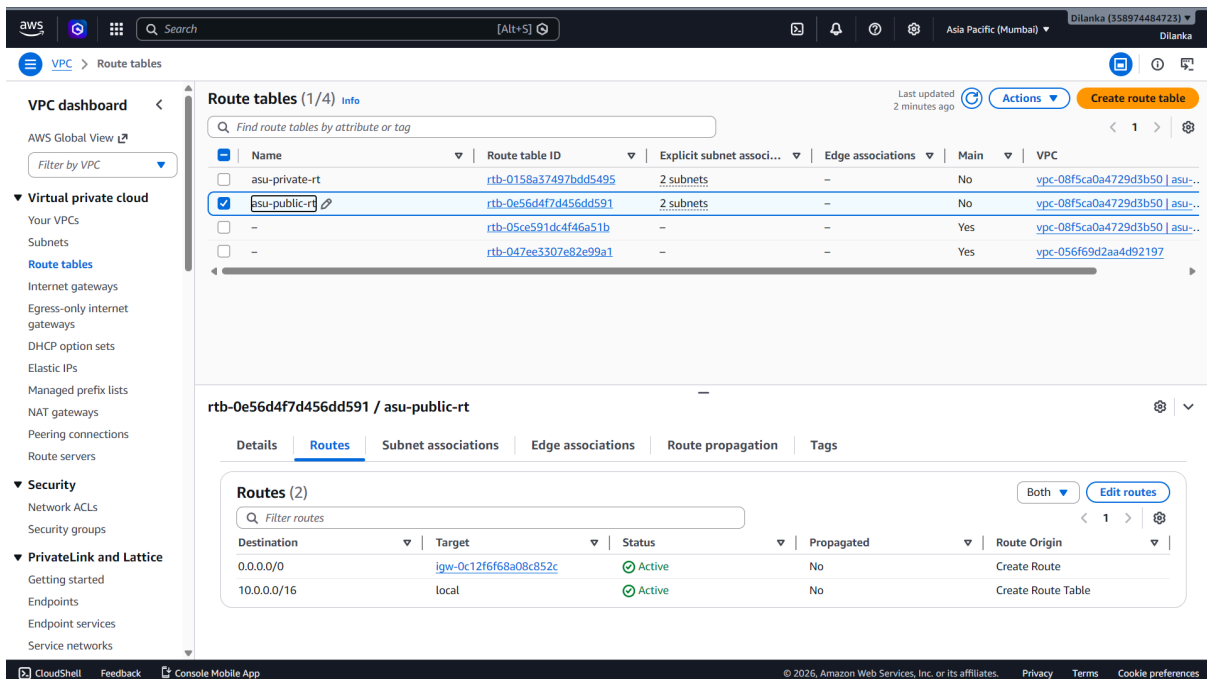


Figure 10: Practical Implementation - VPC Creation (2/2).

10.2. Lambda, API Gateway Deployment

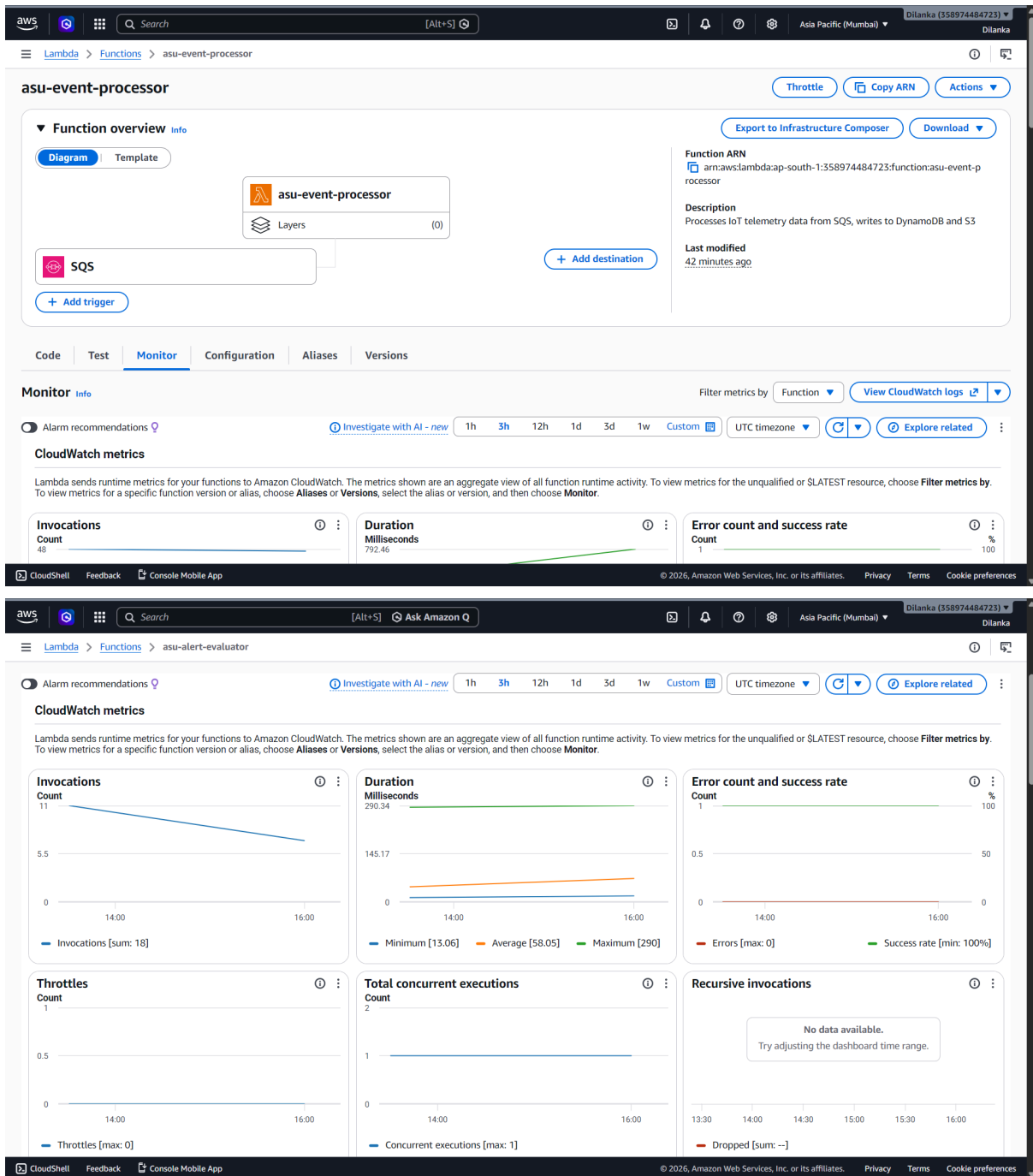


Figure 11: Practical Implementation - Lambda and API Gateway Deployment (1/3).

Figure 12: Practical Implementation - Lambda and API Gateway Deployment (2/3).

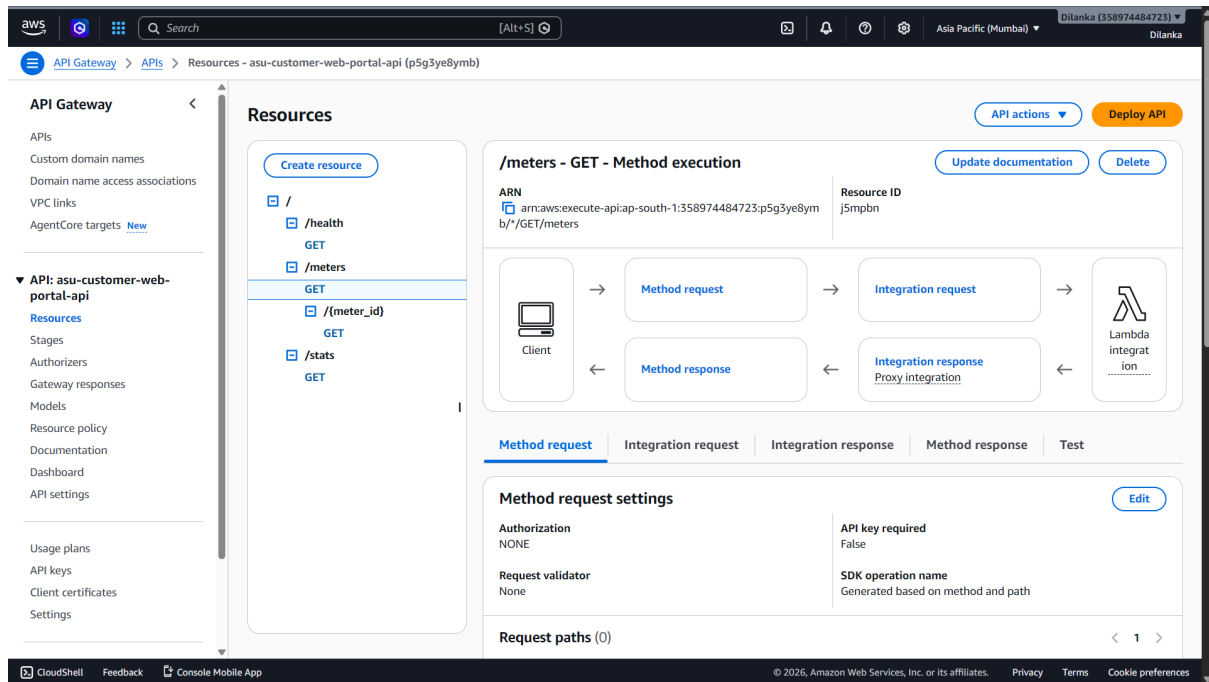


Figure 13: Practical Implementation - Lambda and API Gateway Deployment (3/3).

10.3. IoT device simulation

```
def simulate(args):
    # Run the IoT meter simulation
    iot_client = boto3.client("iot-data", region_name=args.region)

    locations = [
        "mumbai-zone-1", "mumbai-zone-2", "mumbai-zone-3",
        "pune-zone-1", "pune-zone-2",
        "bangalore-zone-1", "bangalore-zone-2",
        "delhi-zone-1", "delhi-zone-2", "delhi-zone-3",
    ]

    print("=" * 60)
    print("AquaSense Smart Meter Simulator")
    print("=" * 60)
    print(f"  Region:  {args.region}")
    print(f"  Meters:  {args.meters}")
    print(f"  Readings: {args.readings} per meter")
    print(f"  Interval: {args.interval}s between readings")
    print(f"  Anomaly:  {'ENABLED' if args.anomaly else 'Disabled'}")
    print(f"  Total messages: {args.meters * args.readings}")
    print("=" * 60)
```

```

total_sent = 0
total_errors = 0

for reading_num in range(1, args.readings + 1):
    print(f"\n--- Reading batch {reading_num}/{args.readings} ---")

    for meter_num in range(1, args.meters + 1):
        meter_id = f"asu-water-meter-{meter_num:04d}"
        location = random.choice(locations)
        topic = f"telemetry/water-meter/{meter_id}"

        # Generate reading (anomaly if flag set, ~20% chance)
        if args.anomaly and random.random() < 0.2:
            reading = generate_anomaly_reading(meter_id, location)
        else:
            reading = generate_normal_reading(meter_id, location)

        try:
            iot_client.publish(
                topic=topic,
                qos=1,
                payload=json.dumps(reading),
            )
            total_sent += 1
            print(f" [{total_sent}] {meter_id} -> {topic} "
                  f"(flow={reading['water_flow_lph']}L/hr, "
                  f"pressure={reading['pressure_psi']}PSI)")

        except Exception as e:
            total_errors += 1
            print(f" [ERROR] {meter_id}: {e}")

    if reading_num < args.readings:
        print(f"\n Waiting {args.interval}s before next batch...")
        time.sleep(args.interval)

print("\n" + "=" * 60)
print(f"Simulation Complete!")
print(f" Total sent: {total_sent}")
print(f" Total errors: {total_errors}")
print("=" * 60)

```

```

Help  ← → ecdwa-project  1  Update
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  SPELL CHECKER  POSTMAN CONSOLE

(.venv) PS D:\apiit-3rd-yr\cloud-assignment\ecdwa-project\iot-simulation> py simulate_meters.py --meters 10 --readings 5 --anomaly

=====
AquaSense Smart Meter Simulator
=====
Region:  ap-south-1
Meters:  10
Readings: 5 per meter
Interval: 2.0s between readings
Anomaly:  ENABLED
Total messages: 50
=====

--- Reading batch 1/5 ---
[1] asu-water-meter-0001 -> telemetry/water-meter/asu-water-meter-0001 (flow=59.51L/hr, pressure=34.79PSI)
[2] asu-water-meter-0002 -> telemetry/water-meter/asu-water-meter-0002 (flow=99.54L/hr, pressure=51.24PSI)
[3] asu-water-meter-0003 -> telemetry/water-meter/asu-water-meter-0003 (flow=26.48L/hr, pressure=33.38PSI)
[ANOMALY] High pressure: 102.84 PSI
[4] asu-water-meter-0004 -> telemetry/water-meter/asu-water-meter-0004 (flow=147.64L/hr, pressure=102.84PSI)
[5] asu-water-meter-0005 -> telemetry/water-meter/asu-water-meter-0005 (flow=118.65L/hr, pressure=61.75PSI)
[6] asu-water-meter-0006 -> telemetry/water-meter/asu-water-meter-0006 (flow=68.38L/hr, pressure=67.96PSI)
[7] asu-water-meter-0007 -> telemetry/water-meter/asu-water-meter-0007 (flow=84.19L/hr, pressure=46.96PSI)
[8] asu-water-meter-0008 -> telemetry/water-meter/asu-water-meter-0008 (flow=78.82L/hr, pressure=61.86PSI)
[9] asu-water-meter-0009 -> telemetry/water-meter/asu-water-meter-0009 (flow=160.26L/hr, pressure=30.82PSI)
[10] asu-water-meter-0010 -> telemetry/water-meter/asu-water-meter-0010 (flow=105.94L/hr, pressure=39.2PSI)

Waiting 2.0s before next batch...

--- Reading batch 2/5 ---
[11] asu-water-meter-0001 -> telemetry/water-meter/asu-water-meter-0001 (flow=120.04L/hr, pressure=50.88PSI)
[12] asu-water-meter-0002 -> telemetry/water-meter/asu-water-meter-0002 (flow=179.68L/hr, pressure=41.51PSI)
[13] asu-water-meter-0003 -> telemetry/water-meter/asu-water-meter-0003 (flow=128.9L/hr, pressure=60.26PSI)
[14] asu-water-meter-0004 -> telemetry/water-meter/asu-water-meter-0004 (flow=164.94L/hr, pressure=53.88PSI)
[15] asu-water-meter-0005 -> telemetry/water-meter/asu-water-meter-0005 (flow=148.8L/hr, pressure=68.85PSI)
[16] asu-water-meter-0006 -> telemetry/water-meter/asu-water-meter-0006 (flow=158.14L/hr, pressure=60.97PSI)
[17] asu-water-meter-0007 -> telemetry/water-meter/asu-water-meter-0007 (flow=67.23L/hr, pressure=35.59PSI)
[18] asu-water-meter-0008 -> telemetry/water-meter/asu-water-meter-0008 (flow=98.27L/hr, pressure=30.72PSI)
[19] asu-water-meter-0009 -> telemetry/water-meter/asu-water-meter-0009 (flow=32.94L/hr, pressure=39.72PSI)
[20] asu-water-meter-0010 -> telemetry/water-meter/asu-water-meter-0010 (flow=26.33L/hr, pressure=49.19PSI)

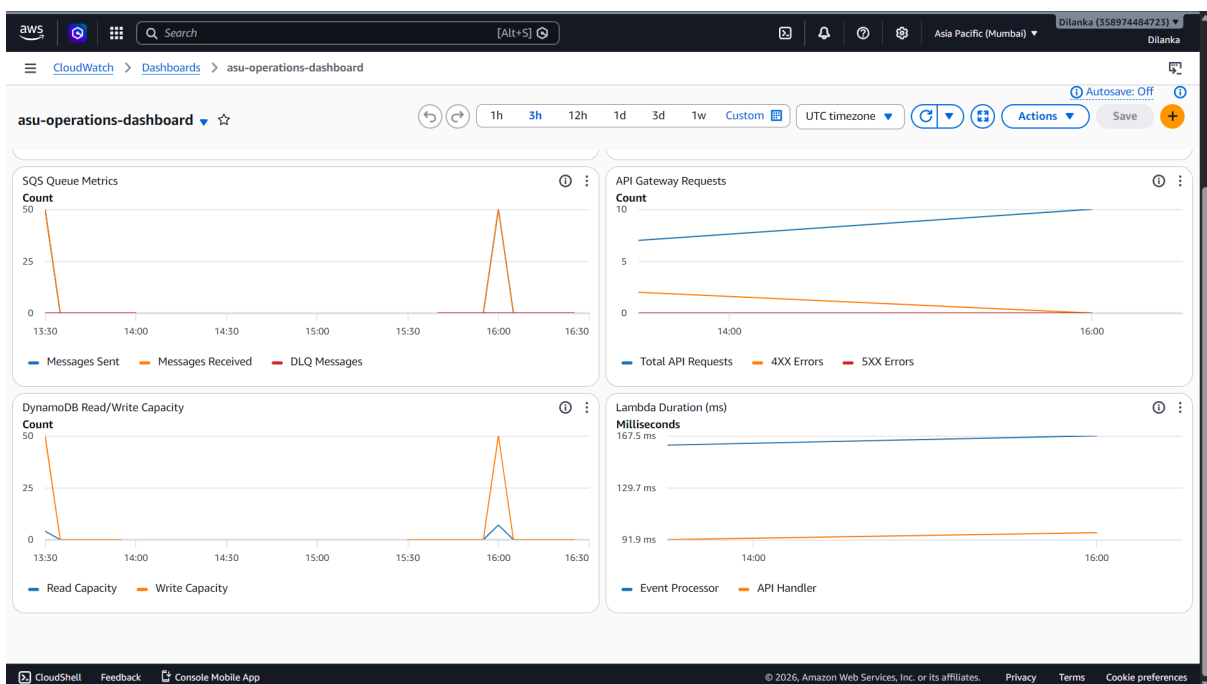
Waiting 2.0s before next batch...

--- Reading batch 3/5 ---
[ANOMALY] Low pressure: 12.28 PSI

```

Figure 14: Practical Implementation - IoT Device Simulation Output.

10.4. CloudWatch metrics and CloudTrail logs



The screenshot shows the AWS CloudWatch Alarms console. The left sidebar contains navigation links for CloudWatch, Alarms, and various monitoring services. The main content area displays a list of alarms under the 'Alarms (3)' heading. The table lists the following alarms:

Name	State	Actions	Actions muted - new	Last state update (UTC)	Condition
asu-api-high-latency	OK	Actions enabled	Warning	2026-05-11 15:43:12	Duration > minutes
asu-dlq-messages	OK	Actions enabled	Warning	2026-05-11 15:42:25	Approximate 1 datapoint
asu-event-processor-errors	OK	Actions enabled	Warning	2026-05-11 15:42:19	Errors > 3

The screenshot shows the AWS CloudWatch Log Groups console. The left sidebar contains navigation links for CloudWatch, Log management, and various monitoring services. The main content area displays a list of log groups under the 'Log groups (5)' heading. The table lists the following log groups:

Log group	Log class	Anomaly d...	Deleti...	Bearer...	Dat...	Sen...	Retenti...
/aws/lambda/asu-alert-evaluator	Standard	Configure	Off	Off	-	-	1 week
/aws/lambda/asu-api-handler	Standard	Configure	Off	Off	-	-	1 week
/aws/lambda/asu-event-processor	Standard	Configure	Off	Off	-	-	1 week
/aws/rds/instance/asu-postgres/postgresql	Standard	Configure	Off	Off	-	-	Never exp...
/aws/sagemaker/NotebookInstances	Standard	Configure	Off	Off	-	-	Never exp...

Figure 15: Practical Implementation - CloudWatch Metrics and CloudTrail Logs (1/2).

Figure 16: Practical Implementation - CloudWatch Metrics and CloudTrail Logs (2/2).

10.5. API Testing

The screenshot displays the 'AquaSense Smart Utilities API - Run results' page. The top navigation bar includes links for Overview, AquaSense Sm, GET Health Check, GET List Meters, GET List Meters with Limit, GET Get Specific Meter, and GET Get Statistics. The main header shows 'Run Again', 'New Run', and 'Share' buttons. Below the header, a summary table provides run details:

Source	Environment	Iterations	Duration	All tests	Errors	Avg. Resp. Time
Runner	none	1	4s 67ms	0	0	356 ms

The 'All' tab shows 0 Passed, 0 Failed, 0 Skipped, and 0 Errors. The 'Console log' tab shows the following test results:

- GET Health Check**: `https://p5g3ye8ymb.execute-api-south-1.amazonaws.com/poc/health`. Status: 200. No tests found.
- GET List Meters**: `https://p5g3ye8ymb.execute-api-south-1.amazonaws.com/poc/meters`. Status: 200. No tests found.
- GET List Meters with Limit**: `https://p5g3ye8ymb.execute-api-south-1.amazonaws.com/poc/meters?limit=5`. Status: 200. No tests found.
- GET Get Specific Meter**: `https://p5g3ye8ymb.execute-api-south-1.amazonaws.com/poc/meters/asu-water-...`. Status: 200. No tests found.
- GET Get Statistics**: `https://p5g3ye8ymb.execute-api-south-1.amazonaws.com/poc/stats`. Status: 200. No tests found.

The 'Response' tab for the 'GET List Meters' test shows the following JSON response:

```
1 {
2   "count": 20,
3   "meters": [
4     {
5       "pressure_psi": "34.79",
6       "location": "mumbai-zone-2",
7       "meter_id": "asu-water-meter-0001",
8       "water_flow_lph": "59.51",
9       "processed_at": "2026-05-11T16:00:19.628153+00:00",
10      "timestamp": "1778515218749",
11      "temperature_c": "30.9",
12      "energy_kwh": "1.44",
13      "battery_pct": "77"
14    },
15    {
16      "pressure_psi": "50.88",
17      "location": "mumbai-zone-2",
18      "meter_id": "asu-water-meter-0001",
19      "water_flow_lph": "120.04",
20      "processed_at": "2026-05-11T16:00:21.572231+00:00",
21      "timestamp": "1778515221510",
22      "temperature_c": "25.9",
23      "energy_kwh": "5.67",
24      "battery_pct": "100"
25    },
26    {
27      "pressure_psi": "12.28",
28      "location": "delhi-zone-1",
```

Figure 17: Practical Implementation - API Testing Result.

10.6. Infrastructure as Code Deployment

```
Apply complete! Resources: 78 added, 0 changed, 0 destroyed.

Outputs:

api_gateway_url = "https://p5g3ye8ymb.execute-api.ap-south-1.amazonaws.com/poc"
api_health_url = "https://p5g3ye8ymb.execute-api.ap-south-1.amazonaws.com/poc/health"
api_meters_url = "https://p5g3ye8ymb.execute-api.ap-south-1.amazonaws.com/poc/meters"
api_stats_url = "https://p5g3ye8ymb.execute-api.ap-south-1.amazonaws.com/poc/stats"
cloudtrail_bucket = "asu-cloudtrail-44e5b647"
cloudwatch_dashboard_url = "https://ap-south-1.console.aws.amazon.com/cloudwatch/home?region=ap-south-1#dashboards:name=asu-operations-dashboard"
dynamodb_table_name = "asu-telemetry-data"
iot_endpoint = "a2irqlzf4rw85a-ats.iot.ap-south-1.amazonaws.com"
iot_topic_rule = "asu-telemetry-to-sqs"
lambda_alert_evaluator = "asu-alert-evaluator"
lambda_api_handler = "asu-api-handler"
lambda_event_processor = "asu-event-processor"
private_subnet_ids = [
    "subnet-0f53f25d4102a3cf7",
    "subnet-0f0a8f213ab804b32",
]
public_subnet_ids = [
    "subnet-0efa49e0f1262cdc4",
    "subnet-003c52c4eb5d459e5",
]
rds_endpoint = "asu-primary-db.chog8y60w92a.ap-south-1.rds.amazonaws.com:5432"
s3_data_lake_bucket = "asu-data-lake-44e5b647"
s3_log_archive_bucket = "asu-log-archive-44e5b647"
sagemaker_notebook_name = "asu-anomaly-detection-nb"
sagemaker_notebook_url = "https://ap-south-1.console.aws.amazon.com/sagemaker/home?region=ap-south-1#/notebook-instances/asu-anomaly-detection-nb"
sns_event_topic_arn = "arn:aws:sns:ap-south-1:358974484723:asu-event-topic"
sns_notification_topic_arn = "arn:aws:sns:ap-south-1:358974484723:asu-notification-topic"
sqs_telemetry_queue_url = "https://sqs.ap-south-1.amazonaws.com/358974484723/asu-telemetry-queue"
vpc_id = "vpc-08f5ca0a4729d3b50"
PS D:\apiit-3rd-yr\cloud-assignment\ecdwa-project\infrastructure>

PS D:\apiit-3rd-yr\cloud-assignment\ecdwa-project\infrastructure> terraform output
api_gateway_url = "https://p5g3ye8ymb.execute-api.ap-south-1.amazonaws.com/poc"
api_health_url = "https://p5g3ye8ymb.execute-api.ap-south-1.amazonaws.com/poc/health"
api_meters_url = "https://p5g3ye8ymb.execute-api.ap-south-1.amazonaws.com/poc/meters"
api_stats_url = "https://p5g3ye8ymb.execute-api.ap-south-1.amazonaws.com/poc/stats"
cloudtrail_bucket = "asu-cloudtrail-44e5b647"
cloudwatch_dashboard_url = "https://ap-south-1.console.aws.amazon.com/cloudwatch/home?region=ap-south-1#dashboards:name=asu-operations-dashboard"
dynamodb_table_name = "asu-telemetry-data"
iot_endpoint = "a2irqlzf4rw85a-ats.iot.ap-south-1.amazonaws.com"
iot_topic_rule = "asu-telemetry-to-sqs"
lambda_alert_evaluator = "asu-alert-evaluator"
lambda_api_handler = "asu-api-handler"
lambda_event_processor = "asu-event-processor"
private_subnet_ids = [
    "subnet-0f53f25d4102a3cf7",
    "subnet-0f0a8f213ab804b32",
]
public_subnet_ids = [
    "subnet-0efa49e0f1262cdc4",
    "subnet-003c52c4eb5d459e5",
]
rds_endpoint = "asu-primary-db.chog8y60w92a.ap-south-1.rds.amazonaws.com:5432"
s3_data_lake_bucket = "asu-data-lake-44e5b647"
s3_log_archive_bucket = "asu-log-archive-44e5b647"
sagemaker_notebook_name = "asu-anomaly-detection-nb"
sagemaker_notebook_url = "https://ap-south-1.console.aws.amazon.com/sagemaker/home?region=ap-south-1#/notebook-instances/asu-anomaly-detection-nb"
sns_event_topic_arn = "arn:aws:sns:ap-south-1:358974484723:asu-event-topic"
sns_notification_topic_arn = "arn:aws:sns:ap-south-1:358974484723:asu-notification-topic"
sqs_telemetry_queue_url = "https://sqs.ap-south-1.amazonaws.com/358974484723/asu-telemetry-queue"
vpc_id = "vpc-08f5ca0a4729d3b50"
PS D:\apiit-3rd-yr\cloud-assignment\ecdwa-project\infrastructure>
```

Figure 18: Practical Implementation - Infrastructure as Code Deployment (1/2).

Figure 19: Practical Implementation - Infrastructure as Code Deployment (2/2).

11. Conclusion

The complete AWS migration strategy for AquaSense Smart Utilities (ASU) is effectively overcoming the key constraints of the on-premises infrastructure, specifically scalability, resilience and cost. The proposed transition includes a phased plan of Rehost, Replatform and Refactor, which results in a cloud-native, multi-AZ target architecture. The design utilizes AWS ECS and Lambda to enable a scalable Customer Web Portal (CWP) and a real-time AWS IoT Core-based Smart Meter Data Platform (SMDP) that is integrated with Aurora and DynamoDB. Most importantly, the architecture is secured with strong security features such as AWS WAF, IAM least-privilege policies, GDPR compliant data governance. Significant cost savings, including through managed services and cost optimisation, including Compute Savings Plans and S3 Lifecycle policies, will deliver the business objective of 35% cost savings. In addition, the platform built on Amazon S3 and SageMaker will allow ASU to go beyond monitoring and into the capabilities of anomaly detection and demand forecasting, ensuring 95% availability. This migration puts ASU in front of the curve as a smart utility provider leveraging data.

12. References

Amazon Web Services, 2024. *Overview of Amazon Web Services*. [Online] Available at: <https://docs.aws.amazon.com/pdfs/whitepapers/latest/aws-overview/aws-overview.pdf> [Accessed 11 May 2026].

Amazon Web Services, 2024. *AWS Well-Architected Framework*. [Online] Available at: <https://docs.aws.amazon.com/pdfs/wellarchitected/latest/framework/wellarchitected-framework.pdf> [Accessed 11 May 2026].

Amazon Web Services, 2024. *AWS Cloud Adoption Framework*. [Online] Available at: <https://docs.aws.amazon.com/pdfs/whitepapers/latest/aws-cloud-adoption-framework/aws-cloud-adoption-framework.pdf> [Accessed 11 May 2026].

Amazon Web Services, 2024. *AWS Database Migration Service User Guide*. [Online] Available at: <https://docs.aws.amazon.com/pdfs/dms/latest/userguide/dms-ug.pdf> [Accessed 11 May 2026].

Amazon Web Services, 2024. *AWS IoT Core Developer Guide*. [Online] Available at: <https://docs.aws.amazon.com/pdfs/iot/latest/developerguide/iot-dg.pdf> [Accessed 11 May 2026].

Amazon Web Services, 2024. *AWS IoT Greengrass Version 2 Developer Guide*. [Online] Available at: <https://docs.aws.amazon.com/pdfs/greengrass/v2/developerguide/greengrass-v2-dg.pdf> [Accessed 11 May 2026].

Amazon Web Services, 2024. *Amazon Simple Storage Service (S3) User Guide*. [Online] Available at: <https://docs.aws.amazon.com/pdfs/AmazonS3/latest/userguide/s3-userguide.pdf> [Accessed 11 May 2026].

Amazon Web Services, 2024. *AWS Lambda Developer Guide*. [Online] Available at: <https://docs.aws.amazon.com/pdfs/lambda/latest/dg/lambda-dg.pdf> [Accessed 11 May 2026].

Amazon Web Services, 2024. *Amazon API Gateway Developer Guide*. [Online] Available at: <https://docs.aws.amazon.com/pdfs/apigateway/latest/developerguide/apigateway-dg.pdf> [Accessed 11 May 2026].

Amazon Web Services, 2024. *Amazon CloudFront Developer Guide*. [Online] Available at: <https://docs.aws.amazon.com/pdfs/AmazonCloudFront/latest/DeveloperGuide/cloudfront-dg.pdf> [Accessed 11 May 2026].

Amazon Web Services, 2024. *AWS WAF, AWS Firewall Manager, and AWS Shield Advanced Developer Guide*. [Online] Available at:

<https://docs.aws.amazon.com/pdfs/waf/latest/developerguide/waf-dg.pdf> [Accessed 11 May 2026].

Amazon Web Services, 2024. *AWS Key Management Service (KMS) Developer Guide*.

[Online] Available at:

<https://docs.aws.amazon.com/pdfs/kms/latest/developerguide/kms-dg.pdf> [Accessed 11 May 2026].

Amazon Web Services, 2024. *Amazon Elastic Container Service (ECS) Developer Guide*.

[Online] Available at:

<https://docs.aws.amazon.com/pdfs/AmazonECS/latest/developerguide/ecs-dg.pdf> [Accessed 11 May 2026].

Amazon Web Services, 2024. *Amazon Aurora User Guide*. [Online] Available at:

<https://docs.aws.amazon.com/pdfs/AmazonRDS/latest/AuroraUserGuide/aurora-ug.pdf>

[Accessed 11 May 2026].

HashiCorp, n.d. *Terraform Registry: AWS Provider Documentation*. [Online] Available at:

<https://registry.terraform.io/providers/hashicorp/aws/latest/docs> [Accessed 11 May 2026].